COMPANION LESSON · THE WHOLE HARNESS TOLD AS ONE DAY IN A FACTORY

The new hire who cannot touch anything

A coding agent is easier to understand if you stop thinking about software for a moment and imagine a factory floor. The factory has machines, a locked tool crib, a painted safety line, a supervisor with a clipboard and a notebook in a locker. One new employee arrives, brilliant but strange: they can think about the work, but they have no hands. This lesson follows that employee through a single work order and shows that every part of the harness has an everyday counterpart.

`src/agent.js 215` · `src/tools.js 440` · `src/sandbox.js 360` · `src/hooks.js 360`

CONTENTS

What is in this explainer

00 · The employee with no hands 3

01 · The shift and the work order 4

02 · The first five minutes: setting up the floor 5

03 · The job folder 6

04 · One lap around the floor 7

05 · The telephone call to head office 8

06 · The reply: an answer or a slip 9

07 · The clerk at the tool crib 10

08 · The eight machines 11

09 · The painted line on the floor 12

10 · Permits, banned machines and the cart inspection 13

11 · The nine checkpoint gates 14

12 · The notebook in the locker 15

13 · The radio check and the question budget 16

14 · The next order and the next day 17

15 · The training dummy 18

16 · The whole factory in one table 19

Page numbers refer to the folio at the foot of each page.

00 · The employee with no hands

Imagine a factory hires a brilliant specialist. They can read any document, solve almost any problem and explain their reasoning clearly. There is one rule in their contract: they may never touch a machine, a shelf, a tool or a delivery cart.

So the factory gives them a desk, a pad of requisition slips and a supervisor.

The specialist studies the work order, writes the next thing they want done on a slip, and hands it to the supervisor. The supervisor walks to the machine, performs the action, writes what happened on a receipt, and places the receipt in the job folder. The specialist reads it and writes the next slip.

That is the whole idea. The specialist is the model. The supervisor, the slips and the receipts are the harness. The specialist supplies judgement; the harness supplies hands, memory and rules.

> **The loop is the employee.** Take away the slips and the receipts and you have someone who can only talk about the work, never do it.

01 · The shift and the work order

A factory day is organised around work orders. Each order says one thing: build this, fix that, check the other. The specialist's day is the same.

One work order equals one run of the loop. If the order is "make a small sign and hang it", the specialist does not wander off and repaint the building. They finish that order, hand back an answer, and stop.

A shift is several work orders in a row. The specialist keeps the same desk and the same locker, and the folder from the last order is still open, so the next order can say "now make the letters bigger" and everyone knows which sign is meant.

The factory also has a rule for when the shift ends badly. If the specialist answers, the day is a success. If the phone line to head office breaks, that is a different kind of failure. If nobody is at the gate to hand over an order, that is a third. If the specialist asks for twelve things and still has not answered, that is a fourth. The supervisor records which one happened, because "it did not work" is not a useful end-of-shift report.

> **One order, one loop.** A task that never ends is not ambition, it is a bug.

02 · The first five minutes: setting up the floor

Before the specialist is asked anything, the supervisor prepares the workplace. This preparation is quiet, happens once, and everything later depends on it.

The supervisor reads the gate pass, which lists the options for today: which specialist is on duty, how many questions they may ask, whether commands need a signature, whether the notebook is in use, and what the actual order is. In the program, that is the command line.

Then the supervisor finds the nearest instruction envelope. There may be one in this building and another one up the street; the nearest one wins. It contains the address of head office, the name of the specialist to call, and the house rules. The envelope deliberately beats anything written on the supervisor's sleeve, so an old note in a pocket cannot quietly override today's orders.

Finally the supervisor lays out the physical world: the floor where work happens, the locker with the notebook, the painted safety line, the checkpoint gates, and the telephone that reaches the specialist. Then they print a receipt on the wall.

That receipt matters. It states which floor, which line, which specialist, which rules and which limits are in force. If two people disagree about what happened, the receipt is the answer.

> **Boot is not setup for later.** It is the act of deciding which world this order will live in.

03 · The job folder

Everything the specialist is allowed to know sits in one folder on the desk. The folder has three sections, always in the same order.

The first section is the standing orders: how to work, what the tools are for, what previous shifts learned. This is not a suggestion, it is the first thing the specialist reads every time they sit down.

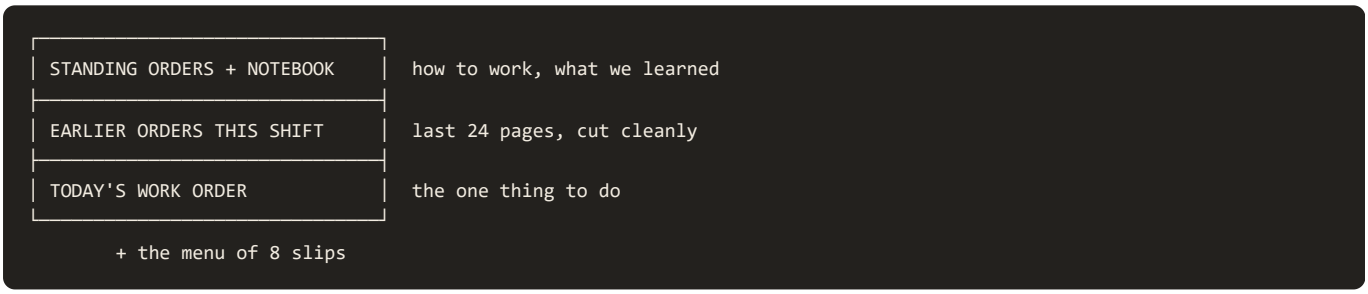
The second section is the old conversation from earlier orders in the same shift. It is trimmed to the last twenty-four pages and always cut at a clean point, never in the middle of a half-finished request.

The third section is today's work order, written by you.

Beside the folder sits a menu: the eight kinds of requisition slip the specialist is allowed to write. The menu is separate from the folder. It is not part of the conversation, it is the shape of what the specialist may ask for.

The specialist has no other memory. If a fact is not in the folder or on the menu, it does not exist for them at this moment. There is no secret drawer.

THE JOB FOLDER · *text*



> **The folder is the mind.** The specialist's entire working state is what is in it, plus what the menu permits.

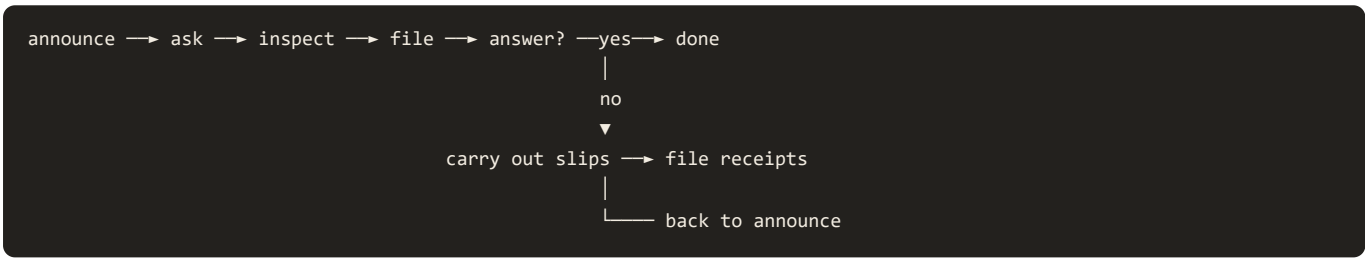
04 · One lap around the floor

The supervisor runs the same seven-step lap for every work order.

First, announce the lap: "question three of twelve", and start the radio check. Second, hand the folder to the specialist and wait for one complete reply. Third, inspect the reply: if it was cut off halfway through a requisition slip, do not act on half a slip, ask for a smaller one. Fourth, file the reply in the folder, keeping only the official note and the slips, not the specialist's private scribbles. Fifth, ask a simple question: did they answer, or did they ask for work? Sixth, if they asked for work, carry out each slip and file the receipt. Seventh, go back to step one with a slightly fatter folder.

That is the entire agent. Everything else in the factory exists to make those seven steps safe, visible and repeatable.

ONE LAP · *text*



The folder only grows during a run. Nothing is thrown away mid-order, which is why the specialist can remember what happened three laps ago.

> **The lap is fixed on purpose.** A predictable loop is easy to watch, easy to test, and hard to break.

05 · The telephone call to head office

One lap contains exactly one telephone call. The supervisor does not send a running commentary; they send the whole folder and the menu, then wait for one complete reply.

Three practical details keep that call sane.

The call is bounded. Each reply may contain at most 2,048 tokens, and the line is cut after a fixed timeout if head office does not pick up. A call that never returns would freeze the whole factory.

The radio check runs while waiting. Every five seconds the supervisor says "still waiting for head office". A slow call must never look like a dead factory.

The reply is cleaned. Only the official note and the requisition slips are kept; the specialist's private reasoning is discarded, because re-sending it on every later lap would waste the folder's space.

The factory deliberately does not stream the answer word by word. It waits for a complete reply, then decides. Streaming would make the console feel livelier, but it would not change the shape of the work.

> **A slow call is not a frozen program.** Say so out loud, every five seconds.

06 · The reply: an answer or a slip

Head office sends back one of two things.

The first is an answer. There are no requisition slips attached. The supervisor prints the answer under a rule and the work order is finished.

The second is one or more slips. Each slip has a name from the menu, a unique serial number, and a form filled in by the specialist. The form is written in JSON, a strict shorthand that the tool crib understands.

A REQUISITION SLIP · *json*

```
{
  "id": "slip_abc",
  "type": "requisition",
  "function": {
    "name": "read_file",
    "arguments": "{\"path\":\"src/agent.js\",\"limit\":40}"
  }
}
```

The form is not trusted. The specialist may name a machine that does not exist, write a number where a word belongs, or ask for a shelf outside the building. Checking that is the next person's job.

There is one absolute rule about slips: every slip must get a matching receipt, even if the machine refused. Head office will reject the next call if a slip is left lying on the desk. That is why filing the receipt happens inside the same lap that runs the machine.

> **Heads it is an answer, tails it is an action.** The loop does not care which; it only checks that every action gets a receipt.

07 · The clerk at the tool crib

Between the specialist and the machines stands one person: the clerk at the tool crib. In the program this role is called the dispatcher. It is a single door, and every request passes through it.

The clerk does four things in order.

First, look up the requested machine. If the name is not on the menu, the clerk writes a polite note listing the machines that do exist, and that note goes back into the folder.

Second, check the form. Is it valid shorthand? Is it a proper form and not a stray sentence? Does each field have the right type? A specialist who writes "twenty" where a number belongs gets a note that says exactly that.

Third, operate the matching machine, using the floor, the form and the run's context.

Fourth, catch everything. A jammed machine never explodes in the clerk's hands. It becomes a receipt that starts with `ERROR`, `FORBIDDEN` or `DENIED`, and that receipt goes into the folder like any other.

The rule that makes the whole factory work: **the clerk always returns a receipt**. Success, refusal, bad form, unexpected smoke, all of it becomes text the specialist reads on the next lap. An exception that escaped would end the shift; a returned receipt is a fact the specialist can recover from.

> **Never throw, always return.** A crash ends the shift; a receipt lets the specialist try again.

08 · The eight machines

The tool crib holds eight machines, and they are not equal.

MACHINE	WHAT IT DOES	GUARD
`list_files`	Reads the shelf labels so you know what exists	Read-only
`read_file`	Reads a slice of a document, with line numbers	Read-only, paginated
`create_file`	Makes a new document, refuses to overwrite	Write fence
`write_file`	Makes or replaces a document	Write fence
`edit_file`	Swaps one exact phrase for another	Write fence, refuses ambiguous matches
`run_command`	Runs a machine and reports exit code, output and errors	Policy, deny-list, cart inspection
`remember`	Writes a line in the locker notebook	Capped and deduplicated
`recall`	Searches the locker notebook	Read-only

Two asymmetries are deliberate.

Reading is open. A specialist who cannot look at a manual, a log or a drawing from another department is useless. Writing is fenced. A specialist who can put things anywhere is dangerous.

And the receipts are deliberately small. Creating a document returns "made it, 42 lines", not the whole document. Editing returns "one change on line 7". The specialist already knows what it asked for. Reading it back to them would waste the one resource that is always short: space in the folder.

> **Read widely, write narrowly, report briefly.** That is the whole tool philosophy.

09 · The painted line on the floor

The specialist is not trusted, so the factory paints a line on the floor around its own bay.

Every save, edit and delete is checked before it happens. The clerk refuses a path that climbs out with two dots, an absolute address outside the bay, a shortcut into someone's home folder, a network address from another building, or anything that resolves outside the painted line.

A refusal is loud and useful. One bold red line on the console, and the same sentence on the receipt the specialist reads. Nothing is written. The specialist learns the boundary from the refusal, not from a stack trace.

This is a guardrail, not a jail. A determined script can still walk out of the building. Actual isolation needs a container with no door. The painted line exists to make the ordinary accident loud and the obvious escape impossible.

> **Fences are best effort.** Say so, and mean it. The goal is that the common mistake is loud and the obvious attack is impossible.

10 · Permits, banned machines and the cart inspection

Writing is not the only way to change a factory. Machines can do it too, so the factory has three more layers of protection.

The first is the permit desk. `CMD_POLICY` is set to `allow`, `ask` or `deny`. In `ask`, the supervisor stops and asks a human to sign for each command. In `deny`, no machine runs at all.

The second is the banned-machines list, which applies no matter what the permit desk says. Recursive deletion of a root path, disk formatting, shutdown, force-pushing shared work, piping the internet straight into a shell, fork bombs, raw disk writes. These are refused even if a human would have signed.

The third is the cart inspection. Before a command runs, the clerk reads the command line and looks for visible destinations: arrows that send output into a file, copy, move, delete, PowerShell destinations. If the destination is outside the painted bay, the command is refused before it starts.

There is a fourth layer that is easy to miss: the environment. A child machine receives thirteen ordinary variables, not the whole building's keys. Without this, a one-line script could print the company's master key into the transcript, and the transcript goes to head office.

> **A command can do what a file tool cannot.** So the command line gets its own inspector.

11 · The nine checkpoint gates

The factory's lap is fixed, but a fixed lap is not extensible. So the floor has nine checkpoint gates where outside staff may watch, annotate, rewrite or stop the work.

GATE	WHEN IT OPENS	WHAT A GUARD MAY DO
`SessionStart`	before the first call	observe, or refuse the shift
`UserPromptSubmit`	as the order arrives	rewrite or block the order
`PreModel`	before every call	inspect the folder, count its size
`PostModel`	after every reply	observe the answer or slip
`PreToolUse`	before every machine	**block** the request
`PostToolUse`	after every machine	**replace** the receipt
`Stop`	the specialist answered	summarise, notify
`StepLimit`	the question budget ran out	observe, escalate
`SessionEnd`	always, finished or not	clean up, write a transcript

A guard returns a small note: nothing means "watching only", a note means "print this", a task means "use this order instead", a replacement means "the specialist should read this instead of the real receipt", and a block means "do not run it, and tell the specialist why".

A guard that misbehaves is reported and ignored. Outside staff must never be able to break the factory's one absolute rule: every slip still gets a receipt.

> **Hooks are gates in a fixed lap.** They add observation and policy without changing the shape of the loop.

12 · The notebook in the locker

The folder on the desk is thrown away at the end of the shift. The notebook in the locker is not.

It is a plain paper notebook with one line per lesson. Each line has a tag, a short sentence and the date it was learned. The tags are `fact`, `pref`, `fix` and `gotcha`.

Because it is paper, a human can read it, copy it, correct it or tear out a wrong page. No viewer, no export step, no locked cabinet.

There are two ways to use it.

- **Eager digest** — the newest twelve lines, capped at 1,200 characters, are copied into the folder before the run starts.
- **Lazy lookup** — the specialist asks `recall` to search the whole notebook while the run is happening.

At the start of every order, the supervisor copies the newest twelve lines, capped at 1,200 characters, onto the front page of the folder. That is the standing order's memory block. It always ends by saying what was left out: nine older notes not shown, search the notebook with `recall`.

During the order, the specialist may ask `recall` to search the whole notebook. That costs one slip, and it only happens when needed.

The notebook is also bounded when written. A duplicate refreshes its date instead of adding a line. One line is capped at 300 characters, one write at 2,000, and the notebook holds at most 60 lines, dropping the oldest.

Why paper instead of a database? Because "why did it remember that?" should be answerable by reading the notebook, not by hunting through a log. No embeddings, no vector search, no automatic summary. The smallest thing that actually survives.

> **Eager digest, lazy recall.** Put the recent essentials in front of the specialist; make the archive one request away.

13 · The radio check and the question budget

Two quiet limits keep a runaway shift from becoming an incident.

The first is the radio check. While the supervisor waits for head office, a timer prints "still waiting" every five seconds. This is not decoration. Without it, a thirty-second network delay looks identical to a crashed program, and an operator kills a run that was about to succeed.

The second is the question budget. A work order allows at most twelve calls to head office. When the twelfth lap ends without an answer, the supervisor stops, fires the step-limit checkpoint, and reports that the budget ran out. The next order in the same shift starts again at one, because a budget is per order, not per day.

Around those two sit smaller caps that matter as much: 2,048 tokens per reply, a fixed request timeout, 6,000 characters per output stream, 200 lines per file read by default, and a 30-second limit on any single command.

Autonomy without limits is not a feature. It is an unbounded bill and an unrepeatable demo.

> **Bounded autonomy is the product.** A visible stop is better than a mystery.

14 · The next order and the next day

A one-shot run does the work once and turns the result into an end-of-shift code: zero for an answer, one for a broken telephone line, two for nobody at the gate to give an order, three for an exhausted question budget.

A shift is different in four ways. The question budget resets for each order. The last twenty-four pages of the folder carry over, so "now add a test" knows what was built a moment ago. The standing orders are rebuilt fresh, so a note saved in the first order is visible in the second. And the folder cut always lands on a clean page, because head office rejects a folder that starts mid-receipt.

The `/new` command clears the conversation but not the project. Files stay on the floor, the notebook stays in the locker. `/quit` ends the shift.

At the end of everything, the floor keeps the documents, the locker keeps the durable facts, and the process ends. The next shift starts with an empty folder and a memory block built from what survived.

> **Conversation is temporary; the floor and the notebook are not.** That split is the whole design.

15 · The training dummy

How do you prove the factory works before you trust it with a real specialist and a real telephone line?

You hire a training dummy. `--demo` replaces exactly one thing: head office. A scripted actor answers the phone and reads a fixed script of requisitions and answers.

The script lists the floor, creates a document, runs it, edits it, runs it again. Then it tries to break out: it asks for a path outside the bay, tries to read the private files next door, tries to write into a home folder, tries to copy something out with the shell, tries a force push, tries to print the master key, and runs a command that fails. Then it writes a note and searches the notebook.

Everything else stays real. The lap, the clerk, the painted line, the permits, the notebook, the checkpoints, the exit codes. The only missing piece is one telephone call.

At the end, a scorecard reads the run's own receipts and checks that each guard actually held. It is a claim the factory makes about itself, printed by the factory.

> **Test the loop, not a mock of the loop.** Replace one dependency and let the real machinery prove itself.

16 · The whole factory in one table

Every part of the harness has a counterpart on the floor. Learn the floor and you have learned the harness.

FACTORY	HARNESS	ITS JOB
The specialist	The model	Reads the folder, writes slips or an answer
The supervisor	`agent.js`	Runs the lap, files receipts, enforces the budget
The job folder	`messages`	The specialist's entire working state
Standing orders	`prompt.js`	How to work and what the tools are for
The menu	Tool schemas	The only shapes a request may take
The clerk	The dispatcher	Validates, runs and always returns a receipt
The eight machines	The eight tools	The only actions in the building
The painted line	`sandbox.js`	Refuses writes outside the bay
The permit desk	`CMD_POLICY`	Asks, allows or denies commands
The cart inspection	Shell-write scan	Refuses commands that write outside
The thirteen badges	Environment allow-list	Keeps the master key out of the transcript
The nine gates	`hooks.js`	Observe, rewrite or block at named moments
The locker notebook	`memory.md`	Facts that survive the shift
The radio check	`heartbeat.js`	Proves a slow call is still alive
The training dummy	`demo.js`	A scripted model with no key and no network
The scorecard	The tests	Proves every guard actually held

> **One sentence:** a specialist asks, a clerk acts, a painted line refuses, a lap repeats, and a notebook remembers, until the specialist answers.

Built from `mini-coding-agent-js`. Every limit in this lesson is the real one: twelve steps, eight tools, nine hooks, 2,048 tokens per reply, thirteen environment variables, a five-second heartbeat and a sixty-entry notebook. Run `npm run demo` to watch the training dummy, and `npm test` to read the scorecard.